Magic-ObjDB:The Poly-Deck Engine

1. Diagrama Conceptual (Resum)

El nucli del sistema es basa en l'**Herència**. No farem una taula plana.

- Tindrem una classe pare abstracta (`Carta`).
- Tindrem classes filles (`Criatura`, `Terra`, `Encanteri`).
- Tindrem un objecte incrustat (`CostMana`) que no és una entitat, sinó part de la carta.
- Tindrem contenidors (`Mazo`, `Jugador`) que agrupen cartes polimòrficament.

---

2. Detall de les Classes i Atributs

Aquest és el llistat que els alumnes han d'implementar al paquet `entitats`.

A. Superclasse (Abstracta)

Aquesta classe no s'instancia directament, però defineix el que tenen en comú totes les cartes.

🔷 **Classe: `Carta`** (`abstract`)

- **Annotacions:** `@Entity`, `@Inheritance(strategy=InheritanceType.JOINED)` (o bé sense estratègia en ObjectDB, ja que ho gestiona natiu, però `JOINED` és bona pràctica JPA).
- **Atributs:**
  - `Long id` (@Id @GeneratedValue) → Identificador únic.
  - `String nom` → Ex: "Drac Shivan".
  - `String descripcio` → El text de la carta (regles).
  - `Raresa raresa` (Enum) → COMUNA, RARA, MÍTICA.
  - `String edicio` → Ex: "Alpha", "Ravnica".
  - `CostMana cost` (@Embedded) → Veure apartat B.

B. Component Incrustat (Embeddable)

Per evitar tenir 5 columnes de colors a la taula de cartes o una taula separada, ho agrupem en un objecte valor.

🔶 **Classe: `CostMana`**

- **Annotacions:** `@Embeddable` (No és `@Entity`, no té ID).
- **Atributs:**
  - `int blanc`
  - `int blau`
  - `int negre`
  - `int vermell`
  - `int verd`
  - `int incolor`

- *Nota:* A la base de dades, aquests camps es guardaran dins del registre de la `Carta`.

C. Classes Filles (Especialització)

Aquí és on l'Orientació a Objectes brilla. Cada tipus té els seus propis camps.

🟢 **Classe: `Criatura`** (Extends `Carta`)

- **Atributs:**
  - `int forca` → Ex: 5 (Mal que fa).
  - `int resistencia` → Ex: 5 (Mal que aguanta).
  - `String tipusCriatura` → Ex: "Goblin", "Elf", "Drac".
  - `boolean vola` → Habilitat simple.

🟤 **Classe: `Terra`** (Extends `Carta`)

- **Atributs:**
  - `Color produccio` (Enum) → Quin color de manà dona (VERMELL, BLAU...).
  - `boolean esBasica` → Si és terra bàsica o especial.

🟣 **Classe: `Encanteri`** (Extends `Carta`)

- **Atributs:**
  - `String tipus` → "Aura", "Maledicció", "Global".
  - `boolean esInstantani` → Si es pot jugar en el torn del rival.

---

3. Detall de les Relacions (Assocacions)

Aquí definim com interactuen les entitats entre elles.

Relació 1: El Mazo (La Llista Polimòrfica)

Un mazo és una llista de cartes. La gràcia és que la llista contindrà barrejats objectes `Criatura`, `Terra` i `Encanteri`.

📦 **Classe: `Mazo`**

- **Atributs propis:**
  - `Long id`
  - `String nom` → Ex: "Goblin Aggro".
  - `LocalDate dataCreacio`
- **Relacions:**
  - **Cartes (N:M):** `@ManyToMany`
    - Tipus: `List<Carta> cartes`
    - *Configuració:* `CascadeType.PERSIST` (però NO `REMOVE`, si esborro el mazo no vull que s'esborrin les cartes de la base de dades general, només la relació).

Relació 2: El Jugador (El Propietari)

Gestiona la propietat.

👤 **Classe: `Jugador`**

- **Atributs propis:**
  - `Long id`
  - `String nick`
  - `int nivell`
- **Relacions:**
  - **Mazos (1:N):** `@OneToMany`
    - Tipus: `List<Mazo> mazos`
    - *Configuració:* `CascadeType.ALL` + `orphanRemoval=true`. (Si esborro el jugador, s'esborren els seus mazos. Si trec un mazo de la llista, s'elimina).
  - **Col·lecció (N:M):** `@ManyToMany`
    - Tipus: `List<Carta> col·leccio`
    - *Explicació:* Representa totes les cartes que té el jugador al seu "àlbum", estiguin o no en un mazo.

---

4. Resum Tècnic per a l'Alumne (Checklist)

Per implementar aquest esquema correctament, l'alumne ha de:

1. **Fer servir `@Inheritance`:** A la classe `Carta`.
2. **Fer servir `@Embeddable` / `@Embedded`:** Per al `CostMana`.
3. **Gestionar la llista `List<Carta>`:** Dins de `Mazo`, entenent que ObjectDB guardarà referències als fills automàticament.
4. **Enumerats:** Crear `public enum Raresa { COMUNA, INFREQÜENT, RARA, MITICA }` per evitar errors de text.

En el context de les Bases de Dades Orientades a Objectes (BDOO) i JPA, la **Inheritance** (Herència) és la capacitat de traslladar l'estructura jeràrquica de les teves classes Java directament a la base de dades.

És un dels punts on les BDOO guanyen per golejada a les bases de dades relacionals (SQL), ja que en SQL l'herència no existeix de forma nativa i s'ha de "simular" amb taules estranyes.

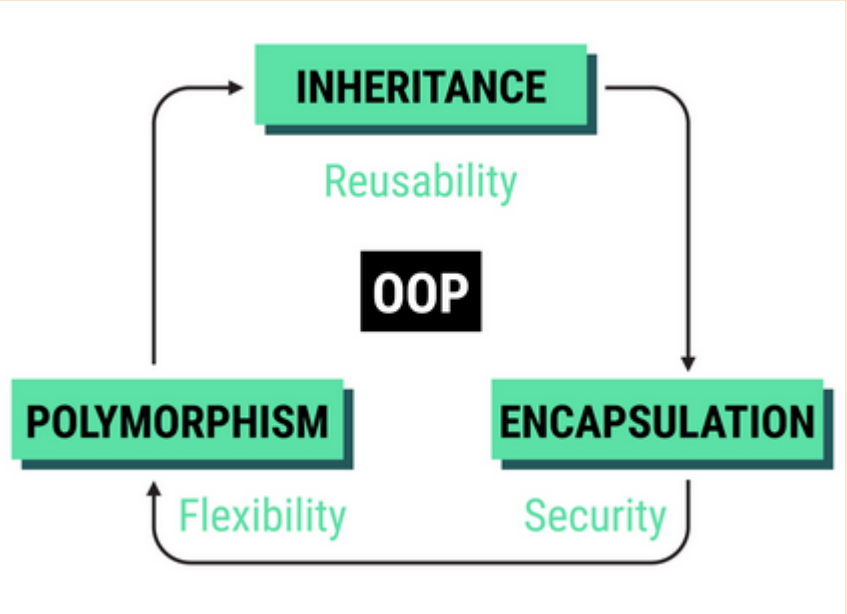
Aquí tens què significa i com funciona segons els teus apunts:

1. El concepte: "És un"

L'herència permet definir una classe genèrica (Pare) i classes més específiques (Filles) que hereten els seus atributs.

- **Exemple Magic:** Una `Criatura` **és una** `Carta`. Una `Terra` **és una** `Carta`.
- Totes les cartes tenen un `nom` i un `cost`, però només les criatures tenen `força` i `resistència`.

2. Els grans avantatges en BDOO (ObjectDB)



L'herència permet dues coses clau:

- **Polimorfisme en les consultes:** Pots fer una consulta a la classe pare i la base de dades et retornarà

tots els fills.
    - *Consulta:* SELECT c FROM Carta c
    - *Resultat:* ObjectDB et retornarà una llista on hi haurà barrejades Criatures, Terres i Encanteris. Java sabrà de quin tipus és cadascuna automàticament.
- **Simplificació del model:** No necessites fer un "JOIN" entre la taula de cartes i la de criatures. Per a tu, com a programador, un objecte Criatura és un sol bloc que ja conté tota la informació de la Carta.

3. Estratègies d'Herència.

Encara que ObjectDB ho gestiona de forma molt eficient, en l'estàndard JPA existeixen tres maneres de guardar l'herència:

1. **Single Table (Una sola taula):** Tots els fills van a la mateixa taula (molts camps queden buits/nulls). És la més ràpida.
2. **Joined (Taules unides):** El pare té una taula i cada fill una altra. És la més neta.
3. **Table per Class:** Cada classe filla té la seva pròpia taula amb tots els camps.

En resum: La *inheritance* significa que la base de dades respecta la teva estructura de classes Java, permetent-te guardar i recuperar objectes complexos mantenint la seva relació de "pare i fill" sense haver de fer servir claus foranes manuals per connectar-los.

4.1. SINGLE_TABLE (Una sola taula)

Concepte: Tot es guarda a la taula CARTA. JPA afegeix una columna extra "invisible" per saber de quin tipus és cada fila (columna discriminadora).

És l'estratègia per defecte de JPA. Imagineu que tenim la classe `Carta` i les filles `Criatura` i `Terra`.

- **Com es guarda:** Es crea una única taula anomenada `CARTA`. Aquesta taula té **totes** les columnes: les del pare (`nom`, `cost`) i les de tots els fills (`força`, `color_produccio`).
- **Columna Discriminadora:** Es crea una columna especial (normalment anomenada `DTYPE`) que diu si aquella fila és una "Criatura" o una "Terra".
- ✅ **Avantatge:** És la més ràpida. Per recuperar una carta no cal fer cap JOIN, només llegir una fila.
- ❌ **Inconvenient:** Molt "bruta". Si una fila és una `Terra`, les columnes de `força` i `resistència` estaran buides (`NULL`). Si tens molts tipus de cartes diferents, tindràs una taula amb centenars de columnes plenes de buits.

```
import javax.persistence.*;

@Entity
// Estratègia: Tot a una taula
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
// Columna 'chivata' que dirà "CRIATURA" o "TERRA"
@DiscriminatorColumn(name = "tipus_carta", discriminatorType = DiscriminatorType.STRING)
public abstract class Carta {
    @Id @GeneratedValue
    private Long id;
    private String nom;
}

@Entity
```

```java
// Valor que es posarà a la columna 'tipus_carta'
@DiscriminatorValue("CRIATURA")
public class Criatura extends Carta {
    private int forca;
    private int resistencia;
}
```

Resultat a la BD (SQL): Taula `CARTA` amb columnes: `id`, `nom`, `tipus_carta`, `forca`, `resistencia`.

Nota: Si la carta és una `Terra`, els camps `forca` i `resistencia` seran `NULL`.

4.2. JOINED (Taules unides)

Concepte: Cada classe té la seva pròpia taula. La taula filla només guarda els seus camps i la ID que la connecta amb el pare.

Aquesta és la més "neta" i la que més agrada als administradors de bases de dades clàssics.

- **Com es guarda:** Es crea una taula per a cada classe.
  - Taula `CARTA`: Guarda `id`, `nom` i `cost`.
  - Taula `CRIATURA`: Guarda l'`id` i la `força`.
  - Taula `TERRA`: Guarda l'`id` i el `color_produccio`.
- **Com es recupera:** Si demanes una `Criatura`, JPA ha de fer un **JOIN** entre la taula `CARTA` i la taula `CRIATURA` usant l'ID.
- ✅ **Avantatge:** No hi ha camps buits. Cada taula té només el que li toca. Respecta la integritat de les dades.
- ❌ **Inconvenient:** És més lenta. Cada vegada que vols una carta filla, la base de dades ha de buscar en dues taules diferents i "enganxar-les".

```java
import javax.persistence.*;

@Entity
// Estratègia: Taules separades unides per ID
@Inheritance(strategy = InheritanceType.JOINED)
public abstract class Carta {
    @Id @GeneratedValue
    private Long id;
    private String nom;
}

@Entity
// Opcional: Personalitzar el nom de la clau forana
@PrimaryKeyJoinColumn(name = "carta_id")
public class Criatura extends Carta {
    private int forca;
    private int resistencia;
}
```

Resultat a la BD (SQL):

- Taula `CARTA`: `id`, `nom`.
- Taula `CRIATURA`: `carta_id` (PK i FK), `forca`, `resistencia`.

Nota: Per recuperar una criatura, la BD ha de fer un `INNER JOIN`.

4.3 TABLE_PER_CLASS (Una taula per classe)

Concepte: La classe pare no té taula pròpia (o no guarda dades dels fills). Cada filla té una taula completa amb TOTS els camps (heretats i propis).

Aquesta és la menys utilitzada, però és bo conèixer-la.

- **Com es guarda:** S'ignora la classe pare com a taula. Es crea una taula per a cada fill que sigui "instanciable".
  - Taula `CRIATURA`: Té `id`, `nom`, `cost`, `força`.
  - Taula `TERRA`: Té `id`, `nom`, `cost`, `color_produccio`.
- ✅ **Avantatge:** Si només busques Criatures, és molt ràpid perquè tota la info (inclosa la del pare) està en una sola taula.
- ❌ **Inconvenient:** El gran problema és el **polimorfisme**. Si fas una consulta tipus `SELECT c FROM Carta c`, la base de dades ha de fer una consulta a totes les taules de fills i ajuntar els resultats amb un `UNION`. És molt poc eficient per a consultes genèriques.

```
import javax.persistence.*;

@Entity
// Estratègia: Cada filla té la seva taula autònoma
@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)
public abstract class Carta {
    @Id @GeneratedValue(strategy = GenerationType.AUTO) // Compte: IDENTITY sol fallar aquí
    private Long id;
    private String nom;
}

@Entity
public class Criatura extends Carta {
    // A la BD tindrà columnes: id, nom, forca, resistencia
    private int forca;
    private int resistencia;
}

@Entity
public class Terra extends Carta {
    // A la BD tindrà columnes: id, nom, color
    private String color;
}
```

Resultat a la BD (SQL):

- Taula `CRIATURA`: `id`, `nom`, `forca`, `resistencia`.

- Taula `TERRA`: `id`, `nom`, `color`.
- No hi ha taula `CARTA` amb dades barrejades.

Nota: Si fas `SELECT c FROM Carta c`, la BD ha de fer un `UNION` de totes les taules.

Resum:

- **SINGLE_TABLE:**Molt ràpida (sense joins).,"Molts NULLs, taula bruta."
- **JOINED:**Dades netes i normalitzades.,Més lenta (fa Joins).
- **TABLE_PER_CLASS:**Fàcil si només consultes fills.,Lenta per consultes polimòrfiques (UNION).

Reflexió.

1. **Cas pràctic:** "Tinc 50 tipus de cartes diferents amb atributs molt variats. Quina estratègia triaries?".

   (**Resposta:** Joined, per no tenir una taula gegant amb milers de NULLs).

2. **Rendiment:** "Quina és l'estratègia més ràpida per a consultes polimòrfiques?".

   (**Resposta: Single Table**, perquè no requereix JOINS ni UNIONS).

Nota sobre ObjectDB: Com que el document diu que ObjectDB és eficient, és perquè ell no "pensa" en taules. Guarda l'objecte sencer en el disc amb un punter al seu tipus. Per a ObjectDB, l'herència és natural, no una simulació.

5. Dades

Aquí tens el contingut per a un fitxer anomenat `cartes.txt`. He preparat 5 cartes per a cada categoria (`Criatura`, `Terra`, `Encanteri`), amb dades realistes i equilibrades per a un joc tipus *Magic*.

Pots copiar aquest bloc i desar-lo com a `cartes.txt` perquè els alumnes el llegeixin i el processin amb un `Scanner` o `BufferedReader`.

Contingut del fitxer: `cartes.txt`

```
# TIPUS | NOM | DESCRIPCIO | RARESA | COST_MANA (Blau,Negre,Vermell,Verd,Blanc,Incolor) | ATRIBUTS_ESPECIFICS
# ---------------------------------------------------------------------------------------------------------
# CRIATURES: Nom | Desc | Raresa | Cost | Força | Resistència | Tipus | Vola
CRIATURA|Drac de Foc|Un drac majestuós que crema tot al seu pas.|RARA|0,0,2,0,0,4|5|5|Drac|true
CRIATURA|Cavaller de l'Alba|Protector del regne amb armadura de plata.|COMUNA|0,0,0,0,2,1|2|2|Humà|false
CRIATURA|Guerrer de la Selva|Coneix cada racó del bosc primigeni.|COMUNA|0,0,0,2,0,1|3|2|Elf|false
CRIATURA|Espectre de la Nit|Apareix quan la lluna és plena i s'emporta les
ànimes.|MITICA|0,3,0,0,0,2|4|3|Espectre|true
CRIATURA|Cranc de Vidre|Una criatura marina amb una closca transparent.|INFREQÜENT|2,0,0,0,0,2|1|5|Cranc|false

# TERRES: Nom | Desc | Raresa | Color_Produccio | Es_Basica
TERRA|Muntanya Volcànica|Una muntanya que fumeja constantment.|COMUNA|VERMELL|true
TERRA|Illa de Cristall|Aigües tan pures que semblen miralls.|COMUNA|BLAU|true
TERRA|Pantà Corrupte|Aigües estancades plenes de malediccions.|COMUNA|NEGRE|true
TERRA|Bosc Ancestral|Arbres que han vist néixer el món.|COMUNA|VERD|true
TERRA|Plana Daurada|Camps infinits de blat sota el sol.|COMUNA|BLANC|true

# ENCANTERIS: Nom | Desc | Raresa | Cost | Tipus | Es_Instantani
ENCANTERI|Bola de Foc|Explosió massiva que danya a l'enemic.|RARA|0,0,1,0,0,2|Global|false
ENCANTERI|Contrahechizo|Anul·la la màgia de l'adversari al moment.|INFREQÜENT|2,0,0,0,0,0|Aura|true
ENCANTERI|Creixement Gegant|La criatura objectiu guanya força temporalment.|COMUNA|0,0,0,1,0,0|Aura|true
ENCANTERI|Judici Diví|Elimina totes les amenaces del camp.|MITICA|0,0,0,0,3,3|Global|false
ENCANTERI|Llum de Curació|Restaura la salut del jugador.|COMUNA|0,0,0,0,1,1|Maledicció|true
```

6. Implementació del Motor de Persistència "ObjectGathering"

Context: Ja tenim el model de classes (`Carta`, `Criatura`, etc.) i el fitxer `cartes.txt`. Ara cal programar la lògica que gestiona aquestes dades seguint els estàndards de JPA i ObjectDB.

6.1. El Gestor de Persistència (EntityManager)

D'acord amb els apunts sobre el **Context de Persistència (Cache L1)**, l'alumne ha d'implementar una classe `GestorCartes` que controli el cicle de vida.

- **Tasca:** Crea un mètode per obrir i tancar l'`EntityManagerFactory` i l'`EntityManager`.
- **Punt clau:** Recorda que la Cache L1 neix amb l'`EntityManager`. Si el tanques, tots els teus objectes passen a estat **Detached**.

6.2. Operació CREATE: Càrrega des de Fitxer

- **Objectiu 1:** Implementar un mètode `importarCartes(String rutaArxiu)`.
- **Requisits:**
  - Llegir el fitxer `cartes.txt` que hem creat abans.
  - Instanciar l'objecte correcte segons el tipus (Criatura, Terra o Encanteri).
  - Utilitzar `em.persist(objecte)` per passar-lo d'estat **New** a **Managed**.
- **Objectiu 2:** A continuació implementa tots els mètodes per crear jugadors i “mazos” i afegeix-ne uns quants a la base de dades.

6.3. Operació READ: Consultes i Identitat

- **Tasca A (ID):** Crea un mètode `buscarPerId(long id)`. L'alumne ha de vigilar de passar el literal correcte (ex: `1L`).

  Punt clau: Comprova la **Garantia d'Identitat**. Si crides dues vegades al `find` pel mateix ID, l'EntityManager t'ha de retornar exactament la mateixa instància (comprovació: `obj1 == obj2`).

- **Tasca B (Consultes):**

  1. Consulta Polimòrfica amb Filtre de Subclasse

  "Mostra el nom de totes les **Criatures** que volen (`vola = true`) i que tenen un cost de manà **Negre** superior a 2."

  Per què és complexa? S’ha de saber que, tot i que consulta sobre `Carta`, pot filtrar per atributs que només existeixen a la filla `Criatura`.

  2. Com fer-ho si vols que el resultat sigui `Carta` (Polimorfisme Pur)

  Si el que vols és buscar entre totes les cartes, però que si són criatures compleixin una condició, hauries d'usar la paraula clau `TYPE` o `TREAT` (però en ObjectDB sol ser més net consultar la filla directament si el filtre és específic).

3. Navegació "Deep Path" i Agregació

"Calcula la **mitjana de força** de totes les criatures que pertanyen als mazos d'un jugador concret (identificat pel seu nick)."

Per què és complexa? Requereix navegar des de `Jugador` -> `Mazo` -> `Carta` (filtrant per instància de `Criatura`) i aplicar la funció `AVG()`.

4. Consulta de Cerca per Component Incrustat (@Embedded)

"Troba tots els **Encanteris** que no costen ni manà **Blau** ni manà **Blanc** (cost = 0), però que tenen un cost **Incolor** superior a 3."

Per què és complexa? Obliga a accedir als atributs de l'objecte `@Embeddable CostMana` des de la consulta principal.

Nota:

> *"Recordeu que en JPQL no estem consultant taules, sinó el graf d'objectes en memòria. Si intenteu fer un JOIN manual com a SQL, podríeu estar complicant la consulta innecessàriament. Aprofiteu la navegació per punts (`jugador.mazo.cartes`)."*

6.4. Operació UPDATE: Dirty Checking i Merge

Cal demostrar que s'entenen els estats del cicle de vida.

- **Tasca A (Managed Update):** Recupera una carta, modifica la seva descripció amb un `setDescripcio()` i fes `commit`. **Sense fer persist!** (Demostració del *Dirty Checking*).
- **Tasca B (Merge):** 1. Recupera una carta i tanca l'EntityManager (`em.close()`). L'objecte ara és **Detached**. 2. Modifica el nom de la carta. 3. Obre un nou EntityManager i intenta guardar els canvis. S' haurà d'usar `em.merge(carta)`.

6.5. Operació DELETE: Removed i Orphan Removal

- **Tasca A:** Crea un mètode per eliminar una carta per ID.
- **Tasca B (Relacions):** Crea un `Mazo`, afegeix-li 3 cartes i guarda'l. Després, elimina una carta de la llista del mazo a Java i comprova si s'esborra de la BD.
- **Punt clau del PDF:** L'alumne ha d'explicar la diferència entre fer `em.remove()` i utilitzar l'anotació `orphanRemoval=true` en la relació del Mazo.